anime-rag

RAG multimodal (Agéntico) sobre el dataset Manga109-s — proyecto individual del curso de NLP.

- **Código:** https://github.com/martin76ec/manga-multi-modal-rag
- **Índice construido (embeddings + metadatos, sin imágenes):** https://huggingface.co/datasets/martin76ec/Manga109-s-indexes

Resumen del proyecto del curso

Tema

RAG / RAG Agéntico. No se construyó con LangChain/LangGraph — esta es una desviación deliberada, respaldada por evidencia, no un descuido. Al evaluar este requisito, avancé lo suficiente en la migración como para probarla de verdad:

- `google/gemma-4-26B-A4B-it`, el modelo con capacidad de visión que este proyecto necesita para que el agente pueda juzgar un panel visualmente (no solo a partir del texto de diálogo), **no usa de forma confiable el tool-calling nativo** — confirmado directamente con la API nativa de HF Inference, sin pasar por LangChain: dado una herramienta registrada, simplemente respondió en texto plano afirmando que no tenía herramientas.
- Conectarlo a `create_react_agent` de LangGraph lo empeoró, no lo mejoró: **alucinó una llamada a herramienta completamente falsa** — una URL externa inventada y datos de panel ficticios — en lugar de llamar a la herramienta real o reportar el fallo.
- Los modelos que sí hacen tool-calling de forma confiable en los proveedores de nivel gratuito a los que tengo acceso (`Llama-3.3-70B-Instruct` , `Qwen2.5-72B-Instruct`) son solo de texto. El único candidato con visión + tool-calling que pude encontrar y usar (`Qwen2.5-VL-32B-Instruct`) también ignoró la herramienta y alucinó una respuesta en texto.

Entonces: ningún modelo disponible para mí hace a la vez visión y tool-calling nativo confiable. El agente construido a mano en `agent.py` (un ciclo estilo MRKL `Thought → Action → Observation → Final Answer`, instruido en texto plano en lugar de mediante function-calling nativo, con lógica de reintento/detección de degeneración agregada después de encontrar fallos reales del lado del proveedor — ver Conclusiones) no es una solución alternativa por no conocer LangChain; es el resultado de probar la ruta de LangChain y encontrarla menos confiable para este modelo en particular. Con gusto puedo discutir la investigación completa (evidencia arriba) en la sesión de preguntas.

Problema y datos

Problema: el diálogo del manga en Manga109-s está solo en japonés, y los paneles llevan información visual (quién está presente, qué sostiene, qué está pasando) que el texto de diálogo transcrito a menudo no describe. El objetivo es permitir que alguien que no lee japonés haga una pregunta en

lenguaje natural en inglés y obtenga una respuesta fundamentada en paneles de manga citados — usando tanto el arte como el diálogo, no solo el texto.

Datos: Manga109-s (con acceso restringido, hal-utokyo) — 87 volúmenes de manga con licencia comercial, con imágenes de página y anotaciones por panel (cajas de viñeta, cajas de rostro/cuerpo, texto de diálogo transcrito, nombres de personajes). Completamente indexado para este proyecto: **87 libros, 82,124 paneles**.

Línea base vs. solución propuesta

	Línea base	Propuesta
Recuperación	una sola modalidad: similitud coseno CLIP solo texto **o** solo imagen	puntaje multimodal combinado (`0.6 x imagen + 0.4 x texto` , ajustable) sobre los mismos embeddings de CLIP
Idioma de consulta	solo japonés (tokenizer CLIP crudo)	inglés o japonés (puente de traducción NLLB-200 en ambas direcciones)
Generación de respuesta	se devuelven directamente los top-k paneles, sin razonamiento	agente MRKL+IRCoT: recuperación en varias rondas guiada por el propio razonamiento del modelo, con fundamento visual (ve las imágenes reales del panel, no solo descripciones traducidas), cita libro/página, y dice explícitamente "no lo sé" en lugar de adivinar cuando la evidencia es insuficiente

Metodología de comparación y resultados

No se construyó un benchmark formal con etiquetas (hit-rate@k sobre una muestra aleatoria de consultas, o RAGAS) para esta entrega — esto es una limitación declarada, no un descuido (ver Conclusiones). Lo que sigue es una comparación fundamentada en consultas de prueba reales durante el desarrollo, cada una verificada manualmente contra la imagen real del panel, no solo aceptada sin comprobar:

Ablación de modalidad (misma consulta, `"police officer"` , aislando cada embedding):

Modo	Mejor resultado (top-1)	Puntaje	¿Verificado correcto?
Solo imagen (`w_img=1.0, w_text=0.0`)	`AisazuNihaIrarenai_007`	0.256	no verificado de forma independiente — sin texto de diálogo asociado, señal visiblemente más débil
Solo texto (`w_img=0.0, w_text=1.0`)	`AisazuNihaIrarenai_057`	0.740	**sí** — el panel literalmente dice "ちょっと警察の方!!" ("¡Espera... oficial!!")
Combinado (por defecto `0.6/0.4`)	`AisazuNihaIrarenai_057`	0.403 (= `0.6x0.178 + 0.4x0.740` , verificado a mano)	**sí**, el mismo panel correcto, consistente aritméticamente

Aislar cualquiera de las dos modalidades cambia el ranking; para esta consulta el texto llevó más señal, pero la combinación es lo que un valor por defecto de propósito general necesita para ser robusto en consultas donde ocurre lo contrario (por ejemplo, una consulta puramente visual como "alguien sosteniendo un arma", ver abajo).

Recuperación base vs. agente (consultas de prueba durante el desarrollo, lectura cualitativa de "tarea resuelta"):

Consulta	Línea base (búsqueda directa)	Agente (MRKL+IRCoT)
"police officer"	top-1 correcto, de una sola vez	igual, 1 ronda
"What happens when a police officer shows up at someone's house?"	el usuario necesitaría ya saber que debe buscar "police officer"	encuentra y cita el mismo panel sin indicación previa, sintetiza una respuesta fundamentada sobre la trama del hijo desaparecido
"someone holding a gun" (solo 2 libros indexados, ninguno con un panel real de arma)	devuelve resultados que parecen plausibles pero son incorrectos (no hay arma realmente visible)	**reporta correctamente que no puede encontrar una coincidencia**, en lugar de presentar un resultado incorrecto como correcto
"someone holding a gun" (después de indexar `DollGun`, donde sí existe una coincidencia real)	encuentra el panel correcto, sin explicación	encuentra y **confirma visualmente** el panel ("un primer plano de una pistola Glock"), un detalle que el texto de diálogo no menciona
"The police visit a house about a missing son. What is the father's name, and what does he say about being a parent?"	no puede responder preguntas de múltiples pasos en absoluto (una consulta, una lista ordenada)	ejecutó 4 rondas distintas de recuperación con consultas de seguimiento auto-generadas; encontró correctamente la identidad del padre, luego **admitió honestamente** que no podía confirmar sus palabras exactas sobre la paternidad en lugar de adivinar (la línea correspondiente existía pero quedó mal traducida hasta ser irreconocible — una limitación de calidad de traducción, no un fallo de razonamiento)

Lectura de esto: la principal ventaja medible del agente sobre la línea base es decir correctamente "no lo sé" en lugar de devolver con confianza un top-1 incorrecto cuando la evidencia genuinamente no está presente, y encadenar varias rondas de consultas auto-generadas para preguntas que el usuario no podría haber formulado como un único buen término de búsqueda. Su costo (siguiente sección) es considerable, así que no es una mejora estricta — es un punto diferente en la disyuntiva precisión/costo, apropiado cuando una pregunta no puede responderse con una sola recuperación.

Comparación de costos

	Línea base (búsqueda directa)	**Propuesta (agente)**
Dónde se ejecuta	completamente local (CPU/GPU)	recuperación local; el razonamiento llama a la API de HF Inference (depende de la red, proveedor externo)
Llamadas al LLM por pregunta	0	1 por ronda, hasta `max_steps=4` (+1 respuesta final forzada) = hasta 5
Tokens por llamada	0	`max_tokens=250` de salida, más todo el historial de la conversación (todas las Observaciones previas) reenviado en cada ronda
Imágenes enviadas al modelo	0	hasta `top_k=3` paneles/ronda x hasta 4 rondas, redimensionadas a 384px/JPEG calidad 80 antes de enviarlas
Latencia observada	menos de un segundo	decenas de segundos para una pregunta de varias rondas
Costo monetario/ de cuenta	ninguno	uso del proveedor de HF Inference (se usó el nivel gratuito aquí; se alcanzó un límite de tasa 429 una vez durante las pruebas)

Cómputo/almacenamiento local (costo único, ambos modos lo comparten):

	Valor
Paneles indexados	82,124 (87 libros)
Rendimiento de embedding	~58 paneles/seg (RTX 4060 Ti compartida/con contención, 8GB)
Modelos locales	`line-corporation/clip-japanese-base` (~790MB), `facebook/nllb-200-distilled-600M` (~2.4GB) — ambos gratuitos, locales, sin clave de API
Tamaño del índice construido	380.5MB (un solo archivo Parquet, embeddings + metadatos, sin imágenes)
Datos crudos en disco (imágenes + recortes, dataset completo)	~7.8GB

Conclusiones

Lo que funcionó:

- Combinar embeddings de imagen y texto efectivamente cambia los resultados de recuperación respecto a usar cualquiera de los dos solo — verificado a mano, no asumido (ver tabla de ablación).
- El puente de traducción (NLLB-200, en ambas direcciones) hace que el sistema sea usable por alguien que no lee japonés, tanto al consultar como al mostrar resultados, sin necesitar clave de API (completamente local).

- La capacidad de visión del agente no es cosmética: encontró y describió correctamente el contenido de un panel ("una pistola Glock") que el texto de diálogo nunca mencionó.
- El comportamiento de abstención del agente ("no puedo encontrar esto") se mantuvo consistentemente en lugar de alucinar una respuesta incorrecta pero plausible, que es el comportamiento que realmente importa para una herramienta fundamentada en citas.

Lo que no funcionó, y tuvo que corregirse en el camino (se deja aquí deliberadamente — esto es lo que "qué funcionó / qué no" debe mostrar):

- CLIP colapsaba el diálogo muy corto o solo de puntuación (p. ej. "……") hacia un embedding genérico, contaminando los rankings con puntajes altos espurios — corregido con un filtro de texto mínimo significativo.
- NLLB perdía palabras clave (perdió "police" por completo) al traducir diálogo de manga tartamudeado ("ちょっ...ちょっと") — corregido agregando búsqueda por haces (beam search) y controles de repetición a la generación.
- El proveedor de HF Inference ocasionalmente devolvía salidas malformadas: una vez una respuesta vacía disfrazada de llamada a herramienta, dos veces dos patrones diferentes de bucle de repetición (repetición simple de una palabra, y una cadena repetida con plantilla pero variando su contenido, que la detección por frecuencia de palabras inicialmente no detectó). Todo esto requirió reproducción real y encontrar la causa raíz, no solo envolver el síntoma en un reintento.
- El intento de migración a LangChain/LangGraph (ver Tema, arriba) falló por completo para este modelo — un resultado negativo real, no un paso omitido.

Límites de este experimento:

- No se construyó un benchmark cuantitativo formal (hit-rate@k sobre una muestra de consultas representativa y no seleccionada a mano, o puntuación estilo RAGAS); la comparación anterior es real pero de N pequeño y seleccionada durante el desarrollo, no una muestra aleatoria.
- El **recall** de recuperación es el principal modo de fallo, no la precisión: una respuesta correcta que no está en el subconjunto indexado simplemente no se encontrará, y el agente solo puede decirlo correctamente, no inventarla. Esto se evidenció directamente cuando solo 2 de los 87 libros estaban indexados.
- La traducción de nombres de personajes no es confiable (NLLB no tiene un diccionario de nombres específico de manga), lo que puede hacer que una cita citada por el agente sea técnicamente correcta en significado pero incorrecta en el nombre exacto atribuido.
- La confiabilidad del agente depende en parte del comportamiento de proveedores externos de HF Inference, fuera del control de este proyecto (se observaron salidas transitorias malformadas/ degeneradas de un proveedor).

Declaración de uso de IAG

Este proyecto se construyó con asistencia extensa de Claude (Anthropic), usado como colaborador de ingeniería/programación en pareja durante todo el desarrollo: decisiones de arquitectura, implementación, depuración de fallos reales (colapso de traducción, colapso de embeddings, salidas degeneradas del lado del proveedor, la investigación de LangChain/LangGraph descrita arriba), y esta

documentación. Las decisiones de diseño, la dirección de las pruebas y la aceptación de cada cambio fueron mías; la herramienta de IA realizó la implementación e investigación práctica bajo esa dirección, y sus afirmaciones fueron verificadas repetidamente contra la salida real de ejecución y datos reales, en lugar de aceptarse sin comprobar (por ejemplo, cada afirmación de "ya funciona" en el historial del proyecto estuvo respaldada por una prueba reproducida, no solo declarada). Ajusta la redacción de esta sección según la política exacta de divulgación de tu institución antes de entregar.

Presentación (5 min + 3 min de preguntas)

Estructura sugerida para la presentación de 5 minutos: problema/datos (30s) -> tema + el hallazgo de LangChain/LangGraph (45s) -> demo en vivo de `anime-rag ask` (una consulta de texto, luego una consulta al agente que requiera razonamiento de varios pasos) (2 min) -> tabla de comparación + tabla de costos (1 min) -> conclusiones/límites (45s). Se espera que la sesión de preguntas indague directamente sobre el código y las decisiones tomadas — las secciones anteriores están escritas para ser defendibles punto por punto, no solo descriptivas.

Configuración

```
uv sync
hf auth login                # necesario una vez, para el dataset con acceso
        restringido
```

Manga109-s es un dataset con acceso restringido — solicita acceso en la página del dataset primero.

Uso

```
# listar los 87 libros disponibles
uv run anime-rag books

# extraer + generar embeddings de algunos libros (obtiene solo esos libros del
# zip del dataset, mediante lecturas HTTP por rangos — no hace falta descargar
# el archivo completo de ~3GB)
uv run anime-rag index AisazuNihaIrarenai ARMS

# buscar en inglés — la consulta se traduce automáticamente al japonés antes
# de generar el embedding (el modelo CLIP y el diálogo son ambos nativos de
# japonés), y el diálogo de cada resultado se traduce de vuelta al inglés
# para mostrarlo
uv run anime-rag search "police officer"

# buscar directamente en japonés (omite la traducción, ya que se detecta)
uv run anime-rag search "警察官"

# buscar por imagen — encuentra paneles visualmente similares
uv run anime-rag search --image ruta/a/algun_panel.jpg

# ajustar la mezcla de puntaje imagen/texto, guardar los recortes de los
# resultados para inspeccionarlos
uv run anime-rag search "missing son" --w-img 0.4 --w-text 0.6 --save-crops out/

# omitir la traducción por completo (inicio más rápido si consultas en japonés)
uv run anime-rag search "警察官" --no-translate

# REPL interactivo: carga los modelos una vez, luego permite hacer tantas
# preguntas como se quiera. Basado en menú (buscar por texto / por imagen /
# preguntar al agente / salir), salida con colores, y una miniatura en
# verdadero color del panel del mejor resultado
uv run anime-rag ask

# preguntas y respuestas agénticas: un agente MRKL+IRCoT que genera sus
# propias consultas de recuperación en varias rondas y sintetiza una
# respuesta citada (necesita acceso a HF Inference)
uv run anime-rag agent "What happens when a police officer shows up at someone's
        house?"
```

Cómo funciona

- `source.py` abre el archivo zip del dataset mediante `HfFileSystem` y extrae solo las imágenes/anotaciones de los libros solicitados (lecturas HTTP parciales por rangos).
- `annotations.py` parsea el XML de Manga109 de cada libro en páginas de viñetas (paneles), cajas de texto y cajas de rostro/cuerpo de personajes.
- `panels.py` recorta una imagen por viñeta y la asocia con el texto de diálogo y los nombres de personajes cuyas cajas caen dentro de esa viñeta.

- `embedder.py` envuelve el modelo CLIP japonés para generar embeddings de los recortes de panel y del texto en un espacio compartido.
- `translate.py` traduce las consultas en inglés al japonés antes de generar el embedding, y el diálogo de resultados en japonés de vuelta al inglés para mostrarlo, mediante `facebook/nllb-200-distilled-600M` (local, sin necesidad de clave de API).
- `indexing.py` construye y persiste un índice por libro (embeddings + metadatos) bajo `data/index/`.
- `search.py` clasifica los paneles para una consulta mediante una combinación ponderada de similitud coseno de embeddings de imagen y texto. Los paneles cuyo diálogo está vacío o es demasiado corto para generar un embedding significativo (p. ej. "……") recurren a la puntuación solo por imagen.
- `agent.py` implementa un agente de recuperación **MRKL + IRCoT** sobre `search.py`: una transcripción explícita estilo MRKL `Thought → Action → Observation → Final Answer` (`llm.py`, vía la API de HF Inference — la única parte de este proyecto que no es local) impulsa una única herramienta `search_manga`, pero al estilo IRCoT, cada consulta de recuperación es el propio último paso de razonamiento del agente en lugar de una consulta fija, así que puede perseguir preguntas de varios pasos a través de varias rondas de recuperación y aun así fundamentar su respuesta en evidencia citada de libro/página (o admitir que no lo sabe, en lugar de adivinar). El LLM (`google/gemma-4-26B-A4B-it`) tiene capacidad de visión, así que cada Observación incluye las imágenes reales del panel, no solo el diálogo traducido — puede juzgar qué está pasando visualmente en un panel incluso cuando el diálogo no lo describe.

Los datos descargados y los índices construidos residen bajo `data/` (ignorado por git) — la licencia del dataset prohíbe redistribuirlo.

`anime-rag agent` llama a la API de HF Inference (modelo: `google/gemma-4-26B-A4B-it` por defecto) y necesita acceso a red más un token de HF con acceso a inferencia (`hf auth login`); todo lo demás en este proyecto (embeddings, traducción, indexación) se ejecuta completamente de forma local.

Indexar el dataset completo (p. ej. en una GPU más grande)

El dataset tiene 87 libros (82,124 paneles en total una vez indexados todos). Generar los embeddings localmente en una GPU pequeña o compartida toma un tiempo, así que en una máquina más grande (esto se escribió pensando en una H200):

```
# en la máquina más grande: clona este repositorio, luego
uv sync
hf auth login                       # necesita acceso a Manga109-s

# indexar todos los libros de una vez; sube mucho --batch-size en una GPU grande
uv run anime-rag index --all --batch-size 256

# o, como un solo script (también sube el resultado a HF si se le da un repo id):
./scripts/h200_index_all.sh tunombre/manga109-rag-index
```

`--all` abre el zip del dataset una sola vez e indexa todo lo que encuentra mediante `anime-rag books`, omitiendo cualquier libro ya indexado (seguro de volver a ejecutar / reanudar después de una interrupción).

Mover el índice a otra máquina

`push-index` / `pull-index` suben/descargan **solo el índice construido** (embeddings + metadatos) mediante un repositorio de dataset de HF — nunca las imágenes reales de los recortes de panel, que residen en un directorio separado (`data/raw/crops/`) sobre el que la licencia es más estricta. Esto es una salida numérica derivada, análoga a publicar un modelo preentrenado, no una copia del dataset en sí — ver la nota de licencia abajo. Súbelo como **privado** (el índice propio de este proyecto, enlazado arriba, es público por solicitud).

El índice se consolida en un solo archivo `index.parquet` antes de subirlo — Parquet es el formato estándar de un solo archivo para embeddings + metadatos (lo que usa la propia librería `datasets` de HF), y es mucho más fácil de mover que un directorio por libro (87 libros = 348 archivos pequeños de otro modo). `pull-index` descarga ese único archivo y lo vuelve a dividir de forma transparente en la estructura habitual `data/index/<libro>/`, así que nada más cambia.

```
# en la máquina que construyó el índice
uv run anime-rag push-index tunombre/manga109-rag-index

# si antes subiste con una versión anterior de esta herramienta que subía
# las carpetas crudas por libro en lugar de un solo archivo, agrega --clean
# una vez para eliminar esa estructura antigua del repo después de subir el
# archivo parquet
uv run anime-rag push-index tunombre/manga109-rag-index --clean

# en otra máquina
uv run anime-rag pull-index tunombre/manga109-rag-index

# el índice descargado aún no tiene imágenes de recorte -- regenéralas
# localmente (vuelve a extraer las páginas crudas del zip del dataset y las
# recorta; no implica GPU ni inferencia de modelo, así que es económico
# incluso sin la máquina más grande)
uv run anime-rag crops
```

La búsqueda/citas funcionan inmediatamente después de `pull-index` (es pura matemática de embeddings); solo necesitas `anime-rag crops` cuando quieras ver realmente la imagen del panel de un resultado o usar el agente con capacidad de visión.

Nota de licencia

Manga109-s se distribuye bajo su propia licencia (favorable para uso no comercial pero no abierta), que limita cuánto de las páginas de un libro se puede publicar y prohíbe redistribuir el dataset en sí. Ver la página del dataset para más detalles.